2주차 공부 내용 정리

교재 3장 | AI Agent의 정의, LLM과 RAG와 AI Agent 비교 | 팀 알아서할게

2주차에는 교재 3장을 읽고 AI Agent가 무엇인지 정의하고, LLM과 RAG와 AI Agent가 각각 어떤 문제를 해결하는지 비교했다. 세 방식은 서로 대체하는 관계가 아니라 LLM 위에 기능을 하나씩 더해 가는 관계라서, 각 단계에서 무엇이 추가되는지를 중심으로 정리했다.

01 AI Agent의 정의

AI 교과서로 널리 쓰이는 Russell과 Norvig의 책은 에이전트를 센서로 환경을 인식하고 actuator로 환경에 작용하는 존재로 정의한다. 로봇청소기나 게임 AI도 이 정의에 들어가며, 에이전트라는 개념 자체는 LLM보다 훨씬 오래되었다.

지금 말하는 AI Agent는 이 판단 역할을 LLM이 맡는 시스템이다. 사용자가 목표를 주면 LLM이 무엇을 할지 정하고, 도구를 호출해 행동하고, 결과를 보고 다음 행동을 다시 정한다. 정해진 순서대로만 움직이는 프로그램과 달리 실행 경로를 모델이 그때그때 결정한다.

> Anthropic은 2024년 글 Building effective agents에서 LLM과 도구를 미리 짜 둔 코드 경로대로 연결한 시스템을 workflow, LLM이 스스로 과정과 도구 사용을 정하는 시스템을 agent로 구분했다. 같은 도구를 쓰더라도 누가 흐름을 정하느냐에 따라 에이전트인지가 갈린다.

02 AI Agent의 특징

특징	설명
자율성	사람이 단계마다 지시하지 않아도 목표를 보고 다음 행동을 스스로 정한다.
목표 지향	질문 하나에 답하는 데서 끝나지 않고 목표를 이룰 때까지 여러 단계를 이어 간다.
도구 사용	검색, 계산, 코드 실행, 외부 API 호출로 모델이 혼자 못 하는 일을 처리한다.
메모리	앞 단계의 결과와 대화 내용을 저장해 두고 다음 판단에 쓴다.
적응성	도구 실행이 실패하거나 예상과 다른 결과가 나오면 계획을 고쳐 다시 시도한다.

자율성이 높을수록 사람이 할 일은 줄지만 잘못된 행동을 할 위험도 커진다. 그래서 실제 서비스에서는 결제나 삭제처럼 되돌리기 어려운 행동 앞에 사람의 승인을 받는 단계를 두는 경우가 많다.

03 LLM의 특징과 한계

LLM은 대량의 텍스트로 학습한 지식으로 문장을 생성한다. 요약, 번역, 글쓰기, 코드 작성처럼 입력과 출력이 모두 텍스트인 작업은 혼자서도 잘한다.

- 지식이 학습 시점에 고정되어 있어서 최신 정보나 회사 내부 문서를 모른다.

- 모르는 내용도 그럴듯하게 지어내는 환각이 있다.
- 답을 한 번 생성하면 끝나서 실행 결과를 보고 고치는 과정이 없다.
- 파일을 저장하거나 메일을 보내는 것처럼 외부 시스템에 직접 작용하지 못한다.

04 RAG의 구조와 동작 방식

RAG(Retrieval-Augmented Generation)는 질문과 관련된 문서를 먼저 검색하고, 그 문서를 프롬프트에 함께 넣어 LLM이 답하게 하는 방식이다. 2020년 Meta의 Lewis 등이 제안했고, LLM의 지식 한계와 환각을 줄이는 가장 흔한 방법이 되었다.

단계	하는 일
인덱싱	문서를 chunk 단위로 나누고 embedding 모델로 벡터로 바꿔 vector DB에 저장한다.
검색	질문도 벡터로 바꾼 뒤 가까운 chunk를 찾는다. 키워드 검색을 함께 쓰는 hybrid search도 많이 쓴다.
생성	찾은 chunk와 질문을 프롬프트에 넣고 LLM이 근거를 바탕으로 답한다.

RAG를 쓰면 모델을 다시 학습하지 않아도 문서만 바꿔서 지식을 갱신할 수 있고, 답의 출처를 함께 보여 줄 수 있다. 다만 기본 RAG는 검색하고 생성하는 순서가 고정되어 있어서, 첫 검색이 빗나가면 다시 찾아볼 방법이 없다.

05 LLM, RAG, AI Agent 비교

항목	LLM	RAG	AI Agent
지식 출처	학습 데이터	학습 데이터와 검색한 문서	학습 데이터, 문서, 도구 실행 결과
실행 흐름	입력 한 번에 출력 한 번	검색 후 생성, 순서 고정	목표를 이룰 때까지 판단과 행동 반복
외부 행동	없음	문서 검색만 함	API 호출, 코드 실행, 파일 작성 등
흐름 결정	해당 없음	개발자가 정한 파이프라인	LLM이 상황에 따라 결정
잘 맞는 작업	요약, 번역, 글쓰기	사내 문서 질의응답, 고객 상담	자료 조사, 예약, 코드 수정처럼 여러 단계가 필요한 작업
단점	지식 한계와 환각	검색 품질에 답이 좌우됨	비용과 지연이 크고 결과를 예측하기 어려움

작업이 단순할수록 LLM이나 RAG로 충분하고, 에이전트는 단계 수와 비용이 늘어나는 만큼 이점이 있을 때 쓰는 편이 맞다. 정해진 순서로 풀리는 문제를 에이전트로 만들면 느리고 결과도 불안정해진다.

06 RAG와 AI Agent의 경계

검색 도구를 쓰는 AI Agent와 RAG는 둘 다 문서를 찾아 답에 쓰기 때문에 겉으로 보면 비슷하다. 스터디에서도 이 둘을 어떻게 구분할지 의견이 갈렸다.

검색을 언제, 몇 번, 무엇으로 할지를 누가 정하느냐로 구분하면 가장 분명하다. 개발자가 질문마다 한 번 검색하도록 짜 두었다면 RAG이고, 모델이 검색이 필요한지 판단하고 결과가 부족하면 검색어를 바꿔 다시 찾는다면 에이전트에 가깝다. 이렇게 검색 과정을 에이전트가 주도하는 방식을 Agentic RAG라고 부른다.

07 더 생각해 볼 질문

- 에이전트의 자율성은 어디까지 허용해야 하는가? 행동을 되돌릴 수 있는지와 실패했을 때 비용이 얼마나 큰지가 기준이 될 수 있다.
- context window가 수십만 토큰까지 늘어나면 RAG가 필요 없어지는가? 문서 전체를 넣으면 비용과 지연이 커지고 긴 문맥 중간의 정보를 놓치는 문제가 있어서 검색은 계속 쓰일 가능성이 높다.

참고 자료: 서지영, 랭체인 & 랭그래프로 AI 에이전트 개발하기, 길벗 / Russell & Norvig, Artificial Intelligence: A Modern Approach / Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, 2020 / Anthropic, Building effective agents, 2024